

Started on	Tuesday, 5 August 2025, 7:35 PM
State	Finished
Completed on	Tuesday, 5 August 2025, 8:21 PM
Time taken	46 mins 23 secs
Grade	9.81 out of 10.00 (98%)

Question 1

Correct

Mark 1.00 out of 1.00

Cho đồ thị **vô hướng** gồm **5** đỉnh và **9** cung như hình vẽ.

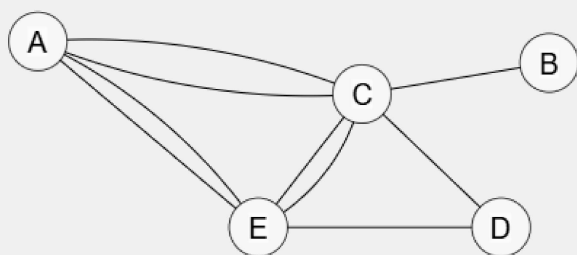
Hãy hoàn thành các bài tập bên dưới.

Answer: (penalty regime: 75, 100, ... %)

Reset answer

Đồ thị gốc (Dùng chuột để thay đổi vị trí của các đỉnh/cung)

Help Clear shift Delete Edit Undo Red Black



1. Loại đồ thị (Graph type)

Đồ thị trên thuộc loại nào?

- ☐ Đơn đồ thị (Simple graph)
- ☒ Đa đồ thị (Multigraph)
- ☐ Giả đồ thị (Pseudograph)

2. Kề nhau (adjacent)

Hai đỉnh u và v được gọi là **kề nhau (adjacent)** $\Leftrightarrow$ có cung (u, v) .

Đánh dấu $\checkmark$ vào các câu đúng, bỏ trống các câu sai.

- ☐ B và E kề nhau.
- ☒ C và E kề nhau.
- ☒ A và C kề nhau.

3. Đỉnh kề/lân cận (adjacent vertices or neighbors)

Đỉnh v là **đỉnh kề/lân cận (adjacent vertex/neighbor)** của đỉnh $u \Leftrightarrow u$ và v kề nhau.

Liệt kê các đỉnh kề của các đỉnh bên dưới, ngăn cách nhau bằng dấu phẩy, bỏ qua các đỉnh trùng nhau.

- Các đỉnh kề của D: C,E
- Các đỉnh kề của C: A,B,D,E
- Các đỉnh kề của B: C

4. Bậc (degree)

Bậc của đỉnh u , kí hiệu $\deg(u)$, là số cung liên thuộc với u (khuyến được tính 2 lần).

Cho biết bậc của các đỉnh bên dưới.

- $\deg(A) =$ 4
- $\deg(E) =$ 5
- $\deg(D) =$ 2

	Test	Got	
✓	Test tự động	1. Loại đồ thị - Ok: Đa đồ thị (multigraph) 2. Kề nhau - B và E: ok. - C và E: ok. - A và C: ok. 3. Đỉnh kề - Các đỉnh kề của D: ok. - Các đỉnh kề của C: ok. - Các đỉnh kề của B: ok. 4. Bậc - Bậc của A: ok. - Bậc của E: ok. - Bậc của D: ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 2

Correct

Mark 1.00 out of 1.00

Cho đồ thị **vô hướng** gồm **5** đỉnh và **7** cung như hình vẽ.

Hãy biểu diễn đồ thị trên bằng phương pháp danh sách cung (edge list).

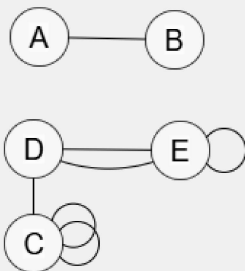
- Liệt kê các cung của đồ thị

Answer: (penalty regime: 10, 20, ... %)

[Reset answer](#)

Đồ thị gốc (Dùng chuột để thay đổi vị trí của các đỉnh/cung cho dễ nhìn)

Help Clear shift Delete Edit Undo Red Black



Danh sách các cung (u, v) của đồ thị trên

- (A , B)
- (C , C)
- (C , C)
- (C , D)
- (E , E)
- (D , E)
- (D , E)

	Test	Got	
✓	Test tự động	Danh sách cung - Cung 1: ok. - Cung 2: ok. - Cung 3: ok. - Cung 4: ok. - Cung 5: ok. - Cung 6: ok. - Cung 7: ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 3

Correct

Mark 1.00 out of 1.00

Vẽ đồ thị **có hướng** có 3 đỉnh và ma trận kề (mở rộng) như sau:

```
0 0 0
0 0 0
0 0 1
```

Chú ý

- Đồ thị có thể có đa cung. Nếu có x cung giữa đỉnh u và v thì ô $(u, v) = x$
- Đồ thị có thể có khuyên. Nếu có khuyên (u, u) , thì ô $(u, u) = 1$

Answer: (penalty regime: 10, 20, ... %)

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

	Test	Expected	Got	
✓	#test	0 0 0 0 0 0 0 0 1	0 0 0 0 0 0 0 0 1	✓

Passed all tests! ✓

Question author's solution (Python3):

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------

Correct

Marks for this submission: 1.00/1.00.

Question 4

Correct

Mark 1.00 out of 1.00

Cho đồ thị **vô hướng** gồm **6** đỉnh và **8** cung như bên như bên dưới.

Hãy áp dụng [thuật toán duyệt đồ thị theo chiều sâu sử dụng ngăn xếp](#) (stack) để duyệt đồ thị trên, bắt đầu từ đỉnh **B**. Với mỗi bước lặp, cho biết đỉnh nào được lấy ra khỏi stack, có làm gì trên đỉnh đó không, những đỉnh nào sẽ được thêm vào stack, nội dung của stack. Ghi các thông tin này vào bảng như sau:

- Cột **u**, ghi đỉnh được lấy ra từ đỉnh stack
- Cột **Duyệt/bỏ qua**, nếu duyệt **u** thì ghi thứ tự của đỉnh được duyệt, thứ tự duyệt tính từ 1. Nếu đỉnh này đã duyệt rồi ghi **bỏ qua**
- Cột **Các đỉnh kề của u**, liệt kê **các đỉnh kề chưa được duyệt của u** theo thứ tự nhỏ đến lớn, ngăn cách với nhau bằng dấu phẩy, ví dụ: A,B,D
- Cột **stack**, liệt kê các đỉnh đang có trong stack, ngăn cách nhau bằng dấu phẩy (đỉnh stack ở phía bên phải), vd: B,C,E. Đỉnh E đang nằm trên đỉnh stack

Quy ước

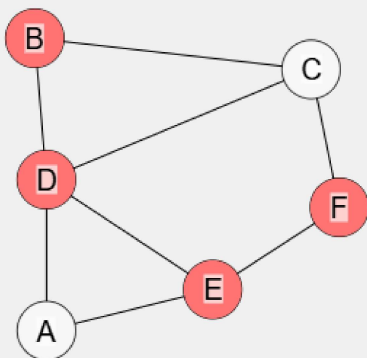
- Đỉnh stack nằm phía tay **PHẢI**.
- Sử dụng thuật toán DFS phiên bản 2 bài tập lý thuyết: đỉnh đã được duyệt sẽ không được thêm vào ngăn xếp nữa.
- Do bản chất của thuật toán, một đỉnh có thể ở trong ngăn xếp nhiều lần.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc

Help Clear shift Delete Edit Undo Red Black



Áp dụng thuật toán duyệt đồ thị theo chiều sâu sử dụng stack và ghi kết quả vào bảng:

	u	Duyệt/bỏ qua	Các đỉnh kề của u	Stack
Khởi tạo				B
1	B	1	C, D	C, D
2	D	2	A, C, E	C, A, C, E
3	E	3	A, F	C, A, C, A, F
4	F	4	C	C, A, C, A, C
5	C	5		C, A, C, A
6	A	6		C, A, C
7	C	bỏ qua		C, A
8	A	bỏ qua		C

	u	Duyệt/bỏ qua	Các đỉnh kề của u	Stack
...	C	bỏ qua		
				Add rowDelete row

	Test	Expected	Got	
✓	1 0 0	Ok	Ok	✓
✓	2 0 0	Ok	Ok	✓
✓	3 0 0	Ok	Ok	✓
✓	4 0 0	Ok	Ok	✓
✓	5 0 0	Ok	Ok	✓
✓	6 0 0	Ok	Ok	✓
✓	2 1 0	Ok	Ok	✓
✓	4 1 0	Ok	Ok	✓
✓	6 1 0	Ok	Ok	✓
✓	2 1 1	Ok	Ok	✓
✓	4 1 1	Ok	Ok	✓
✓	6 1 1	Ok	Ok	✓
✓	4 1 1	Ok	Ok	✓
✓	8 1 1	Ok	Ok	✓
✓	16 1 1	Ok	Ok	✓
✓	24 1 1	Ok	Ok	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Thuật toán Tarjan tìm các thành phần liên thông mạnh (Tarjan's strongly connected components algorithm) dựa trên thuật toán duyệt đồ thị theo chiều sâu (sử dụng đệ quy) kết hợp với đánh số các đỉnh và một ngăn xếp lưu trữ các thành phần liên thông mạnh (strongly connected components gọi tắt là SCC).

```
void SCC(int u) {
    //1. Đánh số cho đỉnh u, đưa u vào ngăn xếp
    num[u] = k; min_num[u] = k; k++;
    push(S, u); on_stack[u] = true;

    //2. Lần lượt xét các đỉnh kề của u
    for (v là các đỉnh kề của u)
        if (v chưa duyệt) {
            //2a. Duyệt v và cập nhật lại min_num[u]
            SCC(v);
            min_num[u] = min(min_num[u], min_num[v]);
        } else if (v còn trên stack) {
            //2b. cập nhật lại min_num[u]
            min_num[u] = min(min_num[u], num[v]);
        } else {
            //2c. bỏ qua, không làm gì cả
        }

    //3. Kiểm tra num[u] == min_num[u]
    if (num[u] == min_num[u]) {
        //Lấy các đỉnh trong stack ra cho đến khi gặp u
        //Các đỉnh này thuộc về một thành phần liên thông
    }
}
```

Thuật toán trên gồm 3 bước chính:

- Đánh số cho đỉnh u, đưa u vào ngăn xếp. Sau khi đánh số xong tăng k lên. Thông thường, k được khởi tạo bằng 1.
- Xét các đỉnh kề v của u, có 3 trường hợp xảy ra:
 - v chưa duyệt => gọi đệ quy duyệt v và cập nhật lại min_num[u]
 - v duyệt rồi nhưng vẫn còn trên stack => cập nhật lại min_num[u]
 - v duyệt rồi và không còn trên stack => bỏ qua
- Kiểm tra num[u] và min_num[u]: nếu bằng nhau thì u là đỉnh khớp (articulation vertex) hay đỉnh cắt (cut vertex)
 - Lần lượt lấy các đỉnh trong stack ra cho đến khi u được lấy ra. Các đỉnh được lấy ra này chính là thành phần liên thông mạnh chứa u.

Cho đồ thị **có hướng** gồm **6** đỉnh và **11** cung như bên như bên dưới. Hãy áp dụng thuật toán Tarjan để tìm các thành phần liên thông mạnh của G bắt đầu từ đỉnh **6**.

Dựa vào kết quả của thuật toán, vẽ các thành phần liên thông tìm được. Với mỗi thành phần liên thông, vẽ các đỉnh và các cung bên trong thành phần liên thông này. Nói cách khác, xóa bỏ các cung của đồ thị gốc mà 2 đỉnh của nó nằm ở 2 thành phần liên thông khác nhau.

Quy ước

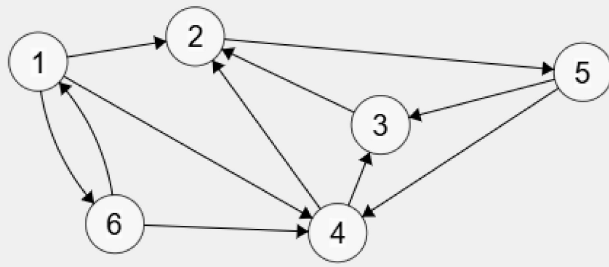
- Mỗi thể hiện của hàm **SCC(u)** được minh họa bằng một khung chữ nhật. Việc gọi đệ quy được minh họa bằng một hình chữ nhật nhỏ hơn bên trong.
- Hãy làm bài theo thứ tự từ ngoài vào trong và từ trên xuống dưới.
- Nếu làm sai 1 bước, có thể bấm "**Lùi lại 1 bước**" để làm lại bước trước đó.
- Liệt kê các đỉnh theo thứ tự **1, 2, 3, ...**
- k được khởi tạo = 1.**

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc





Áp dụng thuật toán Tarjan bằng cách duyệt đệ quy theo chiều sâu bắt đầu từ đỉnh 6

Lùi lại 1 bước Số bước: 40

SCC(6)

1. Đánh số cho đỉnh 6 và đưa nó vào ngăn xếp S.

Gán $\text{num}[6] = \text{min_num}[6] = k = 1$; $k++$;

Đưa 6 vào S. Kết quả: S = 6

Thực hiện đánh số và đưa vào Stack ☒ 1

2. Với các đỉnh kề v của 6: 1,4

Xét ☒ 2

2a. v = '1' chưa duyệt Duyệt nó ☒ 3

SCC(1)

1. Đánh số cho đỉnh 1 và đưa nó vào ngăn xếp S.

Gán $\text{num}[1] = \text{min_num}[1] = k = 2$; $k++$;

Đưa 1 vào S. Kết quả: S = 6,1

Thực hiện đánh số và đưa vào Stack ☒ 4

2. Với các đỉnh kề v của 1: 2,4,6

Xét ☒ 5

2a. v = '2' chưa duyệt Duyệt nó ☒ 6

SCC(2)

1. Đánh số cho đỉnh 2 và đưa nó vào ngăn xếp S.

Gán $\text{num}[2] = \text{min_num}[2] = k = 3$; $k++$;

Đưa 2 vào S. Kết quả: S = 6,1,2

Thực hiện đánh số và đưa vào Stack ☒ 7

2. Với các đỉnh kề v của 2: 5

Xét ☒ 8

2a. v = '5' chưa duyệt Duyệt nó ☒ 9

SCC(5)

1. Đánh số cho đỉnh 5 và đưa nó vào ngăn xếp S.

Gán $\text{num}[5] = \text{min_num}[5] = k = 4$; $k++$;

Đưa 5 vào S. Kết quả: S = 6,1,2,5

Thực hiện đánh số và đưa vào Stack ☒ 10

2. Với các đỉnh kề v của 5: 3,4

Xét ☒ 11

2a. v = '3' chưa duyệt Duyệt nó ☒ 12

SCC(3)

1. Đánh số cho đỉnh 3 và đưa nó vào ngăn xếp S.

Gán `num[3] = min_num[3] = k = 5` ; `k++`;

Đưa 3 vào S. Kết quả: S = 6,1,2,5,3

Thực hiện đánh số và đưa vào Stack ☒ 13

2. Với các đỉnh kề v của 3: 2

Xét ☒ 14

2b. v = '2' duyệt rồi nhưng vẫn còn trên stack Cập nhật `min_num[u]` ☒ 15

Cập nhật `min_num[3] = min(min_num[3], num[2]) = 3` Cập nhật ☒ 16

3. Kiểm tra `num` và `min_num` của 3

`num[3] == min_num[3]` `num[3] != min_num[3]` X 17

Cập nhật `min_num[5] = min(min_num[5], min_num[3]) = 3` Cập nhật ☒ 18

2a. v = '4' chưa duyệt Duyệt nó ☒ 19

SCC(4)

1. Đánh số cho đỉnh 4 và đưa nó vào ngăn xếp S.

Gán `num[4] = min_num[4] = k = 6` ; `k++`;

Đưa 4 vào S. Kết quả: S = 6,1,2,5,3,4

Thực hiện đánh số và đưa vào Stack ☒ 20

2. Với các đỉnh kề v của 4: 2,3

Xét ☒ 21

2b. v = '2' duyệt rồi nhưng vẫn còn trên stack Cập nhật `min_num[u]` ☒ 22

Cập nhật `min_num[4] = min(min_num[4], num[2]) = 3` Cập nhật ☒ 23

2b. v = '3' duyệt rồi nhưng vẫn còn trên stack Cập nhật `min_num[u]` ☒ 24

Cập nhật `min_num[4] = min(min_num[4], num[3]) = 3` Cập nhật ☒ 25

3. Kiểm tra `num` và `min_num` của 4

`num[4] == min_num[4]` `num[4] != min_num[4]` X 26

Cập nhật `min_num[5] = min(min_num[5], min_num[4]) = 3` Cập nhật ☒ 27

3. Kiểm tra `num` và `min_num` của 5

`num[5] == min_num[5]` `num[5] != min_num[5]` X 28

Cập nhật `min_num[2] = min(min_num[2], min_num[5]) = 3` Cập nhật ☒ 29

3. Kiểm tra `num` và `min_num` của 2

`num[2] == min_num[2]` `num[2] != min_num[2]` ☒ 30

Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được 2

Các đỉnh được lấy ra: 4,3,5,2

Nội dung còn lại của ngăn xếp: 6,1

Cập nhật ☒ 31

Cập nhật `min_num[1] = min(min_num[1], min_num[2]) = 2` Cập nhật ☒ 32

2c. v = '4' duyệt rồi và không còn trên stack Bỏ qua X 33

2b. v = '6' duyệt rồi nhưng vẫn còn trên stack Cập nhật `min_num[u]` ☒ 34

Cập nhật $\text{min_num}[1] = \min(\text{min_num}[1], \text{num}[6]) = 1$

Cập nhật ✓ 35

3. Kiểm tra num và min_num của 1

$\text{num}[1] == \text{min_num}[1]$

$\text{num}[1] != \text{min_num}[1]$

X 36

Cập nhật $\text{min_num}[6] = \min(\text{min_num}[6], \text{min_num}[1]) = 1$

Cập nhật ✓ 37

2c. $v = '4'$ duyệt rồi và không còn trên stack Bỏ qua X 38

3. Kiểm tra num và min_num của 6

$\text{num}[6] == \text{min_num}[6]$

$\text{num}[6] != \text{min_num}[6]$

✓ 39

Lấy các đỉnh trong ngăn xếp ra cho đến khi lấy được 6

Các đỉnh được lấy ra: 1,6

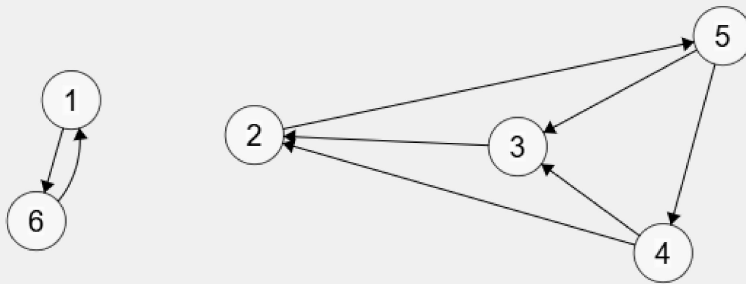
Nội dung còn lại của ngăn xếp:

Cập nhật

✓ 40

Vẽ các thành phần liên thông

Help Clear shift Delete Edit Undo Red Black



	Test	Got	
✓	Test tự động	1. Kiểm tra DFS - Bước 1. SCC(6): 1. Đánh số và đưa u vào ngăn xếp, với u = 6 - Bước 2. SCC(6): 2. Xử lý các đỉnh kề của 6 - Bước 3. SCC(6): 2a. Duyệt 1 - Bước 4. SCC(1): 1. Đánh số và đưa u vào ngăn xếp, với u = 1 - Bước 5. SCC(1): 2. Xử lý các đỉnh kề của 1 - Bước 6. SCC(1): 2a. Duyệt 2 - Bước 7. SCC(2): 1. Đánh số và đưa u vào ngăn xếp, với u = 2 - Bước 8. SCC(2): 2. Xử lý các đỉnh kề của 2 - Bước 9. SCC(2): 2a. Duyệt 5 - Bước 10. SCC(5): 1. Đánh số và đưa u vào ngăn xếp, với u = 5 - Bước 11. SCC(5): 2. Xử lý các đỉnh kề của 5 - Bước 12. SCC(5): 2a. Duyệt 3 - Bước 13. SCC(3): 1. Đánh số và đưa u vào ngăn xếp, với u = 3 - Bước 14. SCC(3): 2. Xử lý các đỉnh kề của 3 - Bước 15. SCC(3): 2b. Cập nhật min_id của 2 - Bước 16. SCC(3): 2b. Cập nhật min_id của 2 - Bước 17. SCC(3): 3b. id[u] != min_id[u] với u = 3 - Bước 18. SCC(5): 2a. Cập nhật min_id của 3 - Bước 19. SCC(5): 2a. Duyệt 4 - Bước 20. SCC(4): 1. Đánh số và đưa u vào ngăn xếp, với u = 4 - Bước 21. SCC(4): 2. Xử lý các đỉnh kề của 4 - Bước 22. SCC(4): 2b. Cập nhật min_id của 2 - Bước 23. SCC(4): 2b. Cập nhật min_id của 2 - Bước 24. SCC(4): 2b. Cập nhật min_id của 3 - Bước 25. SCC(4): 2b. Cập nhật min_id của 3 - Bước 26. SCC(4): 3b. id[u] != min_id[u] với u = 4 - Bước 27. SCC(5): 2a. Cập nhật min_id của 4 - Bước 28. SCC(5): 3b. id[u] != min_id[u] với u = 5 - Bước 29. SCC(2): 2a. Cập nhật min_id của 5 - Bước 30. SCC(2): 3a. id[u] == min_id[u] với u = 2 - Bước 31. SCC(2): 3a. Tìm được SCC của 2 - Bước 32. SCC(1): 2a. Cập nhật min_id của 2 - Bước 33. SCC(1): 2c. Bỏ qua đỉnh 4 - Bước 34. SCC(1): 2b. Cập nhật min_id của 6 - Bước 35. SCC(1): 2b. Cập nhật min_id của 6 - Bước 36. SCC(1): 3b. id[u] != min_id[u] với u = 1 - Bước 37. SCC(6): 2a. Cập nhật min_id của 1 - Bước 38. SCC(6): 2c. Bỏ qua đỉnh 4 - Bước 39. SCC(6): 3a. id[u] == min_id[u] với u = 6 - Bước 40. SCC(6): 3a. Tìm được SCC của 6 2. Kiểm tra các thành phần liên thông Các thành phần liên thông Ok.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 6

Correct

Mark 1.00 out of 1.00

Cho bảng công việc của một dự án như bên dưới.

Hãy vẽ đồ thị mô hình hóa bài toán quản lý dự án này. Vẽ lại đồ thị sau khi đã xếp hạng.

Tính thời gian sớm nhất có thể bắt đầu công việc u: $t[u]$

Tính thời gian trễ nhất có thể bắt đầu công việc u: $T[u]$

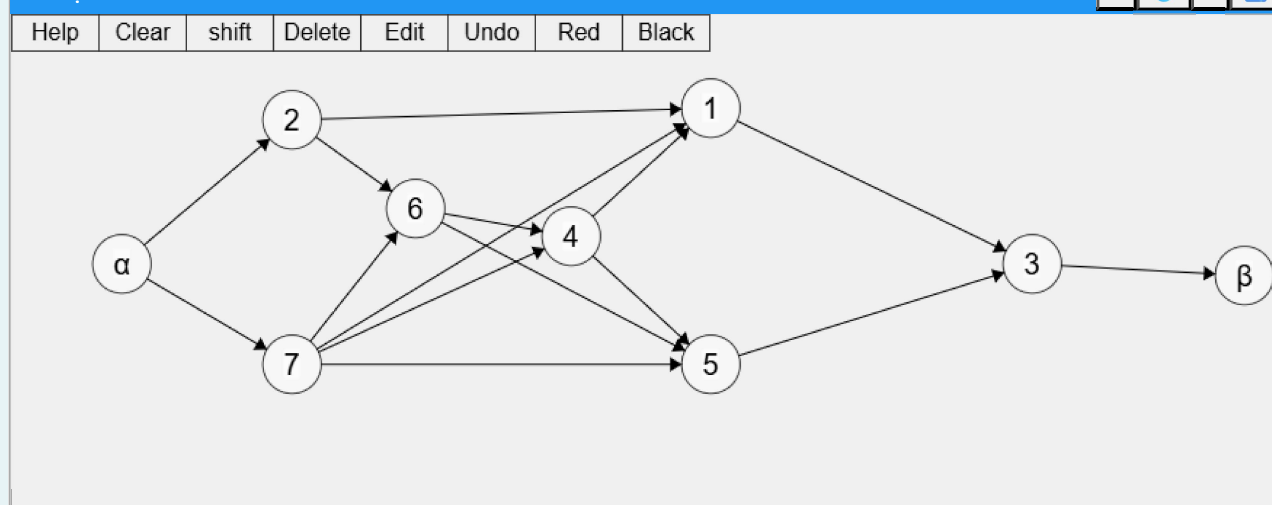
Quy ước

- Hai công việc giả là α và β . Để gõ được ký hiệu này, gõ \alpha và \beta vào định tương ứng.

Answer: (penalty regime: 10, 20, ... %)

Bảng công việc

Công việc	Mô tả	Thời gian hoàn thành (d[u])	Công việc trước đó
1	Công việc 1	2	4, 7, 2
2	Công việc 2	8	
3	Công việc 3	19	1, 5
4	Công việc 4	8	6, 7
5	Công việc 5	16	7, 6, 4
6	Công việc 6	10	7, 2
7	Công việc 7	8	

1. Vẽ đồ thị mô hình hóa bài toán (sắp xếp các đỉnh sao cho đúng với hạng của chúng)**Đồ thị của bài toán****2. Tính $t[u]$ và $T[u]$**

Đánh dấu * vào ô tương ứng với công việc then chốt

	α	1	2	3	4	5	6	7	β
d[u]	0	2	8	19	8	16	10	8	0
t[u]	0	26	0	42	18	26	8	0	61
T[u]	0	40	0	42	18	26	8	0	61
CV then chốt	*		*	*	*	*	*	*	*

	Test	Got	
✓	1. Mô hình hoá về đồ thị (30%) 2. Tính thời gian (70%)	1. Kiểm tra đồ thị & xếp hạng [I] Vị trí đỉnh \alpha ok. [I] Vị trí đỉnh 1 ok. [I] Vị trí đỉnh 2 ok. [I] Vị trí đỉnh 3 ok. [I] Vị trí đỉnh 4 ok. [I] Vị trí đỉnh 5 ok. [I] Vị trí đỉnh 6 ok. [I] Vị trí đỉnh 7 ok. [I] Vị trí đỉnh \beta ok. Tổng (1): 10.00/10. 2. Kiểm tra thời gian 2a Kiểm tra d[u] [I] d[u] của tất cả các đỉnh đều đúng. 2b Kiểm tra t[u] [I] t[u] của tất cả các đỉnh đều đúng. 2c Kiểm tra T[u] [I] T[u] của tất cả các đỉnh đều đúng. 2d Kiểm tra công việc then chốt. [I] Các công việc then chốt Ok. Tổng (2): 12.00/12.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 7

Correct

Mark 0.91 out of 1.00

Cho đồ thị **có hướng** có trọng số không âm gồm **6** đỉnh và **7** cung như bên dưới.

Hãy áp dụng [thuật toán Moore - Dijkstra](#) để tìm các đường đi ngắn nhất từ đỉnh **B** đến các đỉnh khác trên đồ thị. Ở mỗi vòng lặp i ghi lại kết quả trung gian vào các ô tương ứng. Mỗi ô ở cột u ghi hai giá trị $\pi[u]$ và $p[u]$ cách nhau bằng dấu /, ví dụ: cột C được ghi là 4/F thì có nghĩa là $\pi[C] = 4$ và $p[C] = F$.

Dựa vào các $p[u]$ sau cùng, hãy vẽ cây đường đi ngắn nhất. Cây đường đi ngắn nhất gồm tất cả các đỉnh của đồ thị gốc và các cung $(p[u], u)$.

Quy ước

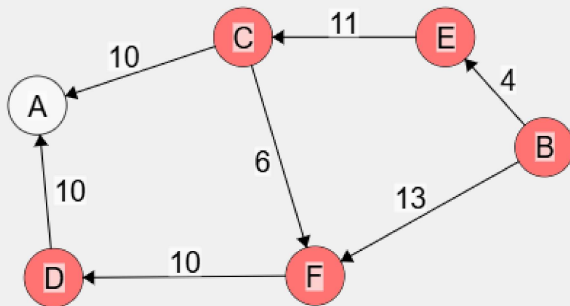
- Sử dụng **oo** (hai ký tự o) để biểu diễn giá trị vô cùng.
- Nếu giá trị $p[u]$ chưa có, có thể bỏ trống, ghi -1 hoặc ghi -
- Đỉnh không cập nhật nữa thì bỏ trống ở cột đó hoặc cũng có thể ghi lại giống hệ hàng bên trên.
- Đánh dấu đỉnh bằng dấu *
- Nếu có 2 đỉnh có cùng giá trị π thì chọn đỉnh có số thứ tự nhỏ.
- Nếu không chọn được u (không có đường đi từ s đến u), thì dừng.
- Cột công việc có thể không ghi.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc (Bấm hoặc dùng chuột để di chuyển đỉnh/cung)

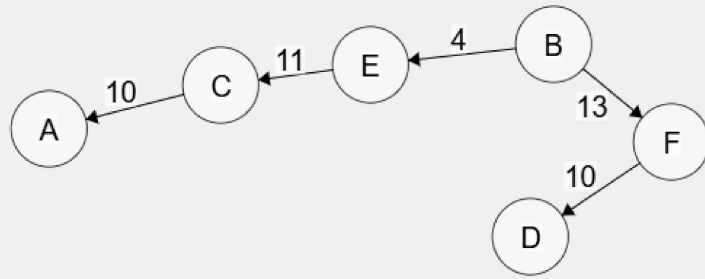
Help Clear shift Delete Edit Undo Red Black



1. Áp dụng thuật toán Moore - Dijkstra và ghi kết quả vào bảng

	A	B	C	D	E	F	Công việc
Khởi tạo	oo/-	0/-	oo/-	oo/-	oo/-	oo/-	
1		*			4/B	13/B	
2			15/E		*		
3				23/F		*	
4	25/C		*				
5				*			

2. Vẽ cây đường đi ngắn nhất



	Test	Got	
✓	1. Thuật toán Moore - Dijkstra (80%) 2. Cây đường đi ngắn nhất (20%)	1. Kiểm tra áp dụng thuật toán Moore - Dijkstra + Bước khởi tạo: - [I] Tất cả các bước con đều đúng. + Bước 1: - [I] Tất cả các bước con đều đúng. + Bước 2: - [I] Tất cả các bước con đều đúng. + Bước 3: - [I] Tất cả các bước con đều đúng. + Bước 4: - [I] Tất cả các bước con đều đúng. + Bước 5: - [I] Tất cả các bước con đều đúng. Tổng (1): 18/18. 2. Kiểm tra cây đường đi ngắn nhất - [I] Cây đường đi ngắn nhất okie. Tổng (2): 11/11.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.91/1.00**.

Question 8

Correct

Mark 1.00 out of 1.00

Cho đồ thị **vô hướng** có trọng số gồm **6** đỉnh và **11** cung như bên như bên dưới.

Hãy áp dụng [thuật toán Kruskal](#) để tìm cây khung vô hướng nhỏ nhất (cây khung có tổng trọng số nhỏ nhất). Ghi kết quả trung gian vào bảng.

Bước 1 (sắp xếp): Sắp xếp các cung theo trọng số tăng dần (đúng ra là không giảm, nhưng nói tăng dần cho dễ nhớ).

- Các cột **u**, **v**, **w** ghi các cung (u, v) và trọng số (w) của chúng theo thứ tự trọng số tăng dần.

Bước 2 (lặp): Lần lượt xét từng cung theo thứ tự đã sắp xếp ở bước 1, với mỗi cung xem xét thêm nó vào cây hay không. Một cung sẽ được thêm vào cây nếu như thêm nó vào không tạo thành chu trình.

Để kiểm tra việc thêm cung có tạo chu trình hay không ta dùng 1 mảng parent để quản lý các bộ phận liên thông của rừng kết quả (kết thúc thuật toán, rừng này sẽ hợp nhất thành cây). Từ một đỉnh u, nếu lần theo parent[u] rồi parent[parent[u]], ... ta sẽ đến đỉnh gốc của bộ phận liên thông (BPLT) chứa u. Khởi tạo, ta gán parent[u] = u với mọi u.

- Cột **root_u** ghi chỉ số của đỉnh gốc của BPLT của u. Cột **root_v** ghi chỉ số của đỉnh gốc của BPLT.
- Nếu root_u = root_v, thì thêm cung (u, v) sẽ tạo thành chu trình. Vì thế ở cột **Thêm vào cây** ta ghi **không** (hoặc **không thêm** hoặc **no**).
- Ngược lại, nếu root_u != root_v, làm các công việc sau:
 - Ở cột Thêm vào cây, ghi **thêm** (hoặc **có** hoặc **x** hoặc **yes**). Khi thêm cung (u, v) vào rừng kết quả, ta phải hợp nhất BPLT chứa u và BPLT chứa v thành một 1 BPLT duy nhất. Để đơn giản, trong bài tập này ta quy ước: **đem gốc của v làm con của gốc u**, có nghĩa là gán **parent[root_v] = root_u** (xem slides bài giảng).
 - Cập nhật lại hình vẽ trong phần **Quản lý các BPLT**.

Bước 3 (Vẽ cây): Dựa vào các cung được chọn thêm vào cây trong bước 2, hãy vẽ cây khung nhỏ nhất trong phần **Cây khung nhỏ nhất**. Cây khung nhỏ nhất gồm tất cả các đỉnh của đồ thị gốc và các cung được thêm vào cây.

Quy ước

- Hai cung có trọng số giống nhau thì ghi cung nào trước cũng được.

Chú ý

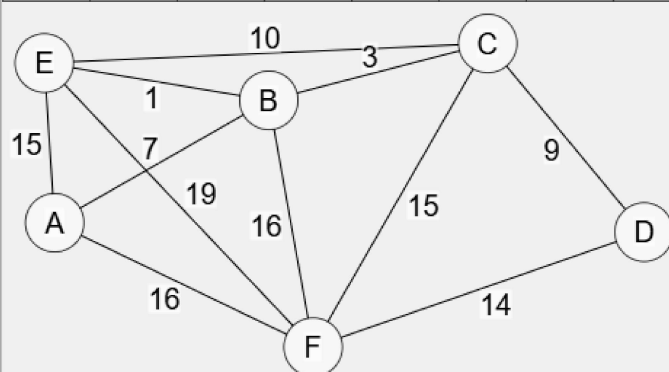
- Cây kết quả phụ thuộc vào thứ tự sắp xếp của các cung.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc (Dùng chuột để thay đổi vị trí của các đỉnh/cung)

Help Clear shift Delete Edit Undo Red Black



Áp dụng thuật toán Kruskal và ghi kết quả vào bảng

	u	v	w	root_u	root_v	Thêm vào cây?
1	B	E	1	B	E	x
2	B	C	3	B	C	x
3	A	B	7	A	B	x

	u	v	w	root_u	root_v	Thêm vào cây?
4	C	D	9	A	D	x
5	C	E	10	A	A	no
6	D	F	14	A	F	x
7	A	E	15	A	A	no
8	C	F	15	A	A	no
9	A	F	16	A	A	no
10	B	F	16	A	A	no
11	E	F	19	A	A	no

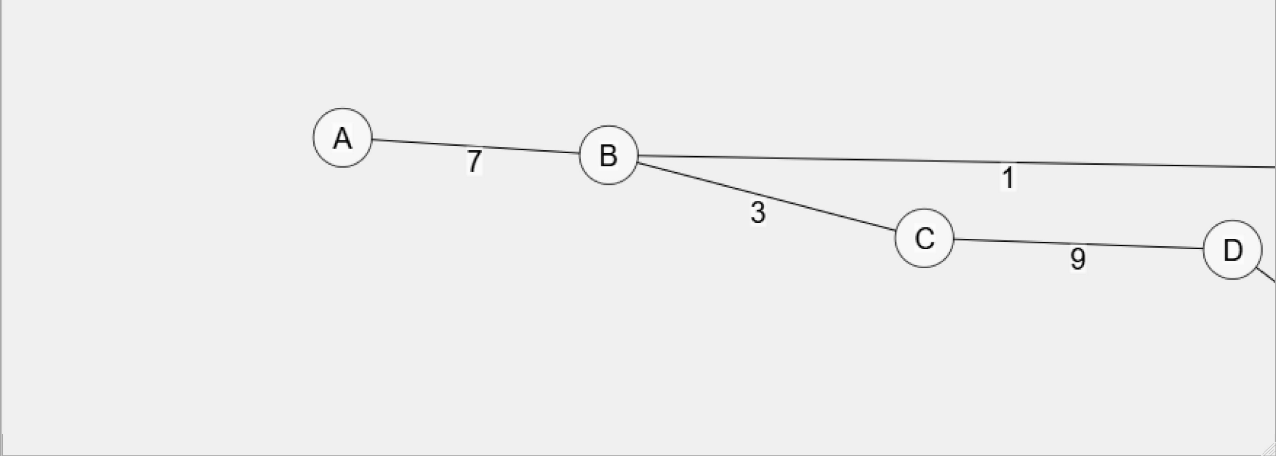
Quản lý các BPLT

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------



Cây khung nhỏ nhất

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------



	Test	Got	
✓	1. Thuật toán Kruskal (80%) 2. Quản lý các BPLT (10%) 3. Cây khung nhỏ nhất (10%)	1. Kiểm tra áp dụng thuật toán a. Sắp xếp các cung + Hàng 1 - [I] cung (B, E) okie. + Hàng 2 - [I] cung (B, C) okie. + Hàng 3 - [I] cung (A, B) okie. + Hàng 4 - [I] cung (C, D) okie. + Hàng 5 - [I] cung (C, E) okie. + Hàng 6 - [I] cung (D, F) okie. + Hàng 7 - [I] cung (A, E) okie. + Hàng 8 - [I] cung (C, F) okie. + Hàng 9 - [I] cung (A, F) okie. + Hàng 10 - [I] cung (B, F) okie. + Hàng 11 - [I] cung (E, F) okie. Tổng (a): 11/11 b. Kiểm tra vòng lặp + Lần lặp 1 - [I] Xử lý cung (B, E) okie. + Lần lặp 2 - [I] Xử lý cung (B, C) okie. + Lần lặp 3 - [I] Xử lý cung (A, B) okie. + Lần lặp 4 - [I] Xử lý cung (C, D) okie. + Lần lặp 5 - [I] Xử lý cung (C, E) okie. + Lần lặp 6 - [I] Xử lý cung (D, F) okie. + Lần lặp 7 - [I] Xử lý cung (A, E) okie. + Lần lặp 8 - [I] Xử lý cung (C, F) okie. + Lần lặp 9 - [I] Xử lý cung (A, F) okie. + Lần lặp 10 - [I] Xử lý cung (B, F) okie. + Lần lặp 11 - [I] Xử lý cung (E, F) okie. Tổng (b): 11/11 Tổng (1): 22/22 2. Kiểm tra quản lý các BPLT [I] Các BPLT okie. Tổng kiểm tra các root: 22/22 Tổng kiểm tra các BPLT: 6/6 Tổng (2): 10.00/10 3. Kiểm tra cây khung nhỏ nhất - [I] Cây khung nhỏ nhất okie. Tổng (3): 10/10 Điểm: 10.00/10	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question 9

Correct

Mark 0.90 out of 1.00

Cho đồ thị **có hướng** có trọng số gồm **6** đỉnh và **10** cung như bên dưới.

Hãy áp dụng thuật toán Chu-Liu/Edmonds tìm cây khung có hướng nhỏ nhất của đồ thị trên.

Quy ước

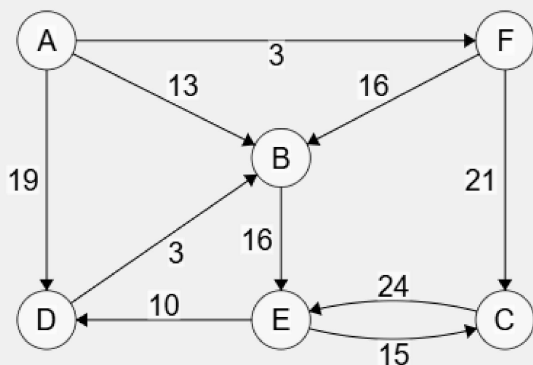
- Đặt tên các đỉnh mới bằng cách ghép tên các đỉnh cũ có trong chu trình. Ví dụ: nếu chu trình gồm các đỉnh B, C và E, thì đỉnh mới sau khi gộp sẽ có tên là **B,C,E**.
- Từ bước G1 trở đi, để dễ truy vết nên ghi nhãn của các cung theo dạng: **9-2 (A, C)** có nghĩa là trọng số mới = trọng số cũ (9) - trọng số của cung đi đến đỉnh trong chu trình (2), và (A, C) là cung trước khi co. Những phần ghi trong dấu ngoặc đơn là chú thích và sẽ được bỏ qua trong quá trình chấm.

Answer: (penalty regime: 10, 20, ... %)

Reset answer

Đồ thị gốc G0 (Dùng chuột để thay đổi vị trí của các đỉnh/cung)

Help Clear shift Delete Edit Undo Red Black



Áp dụng thuật toán Chu-Liu/Edmonds bằng cách vẽ các đồ thị vào các ô tương ứng bên dưới

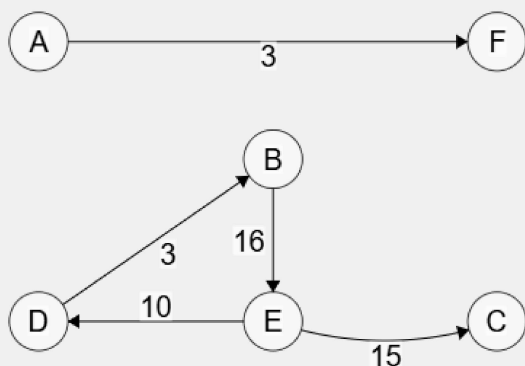
I. Pha co

Lập 0:

- Xây dựng đồ thị xấp xỉ H0: ☐ Copy đồ thị G0

Đồ thị xấp xỉ H0

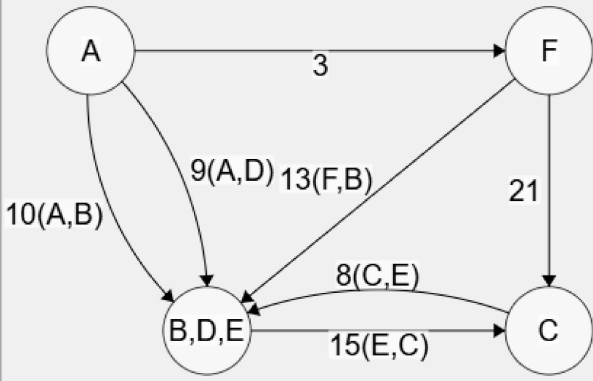
Help Clear shift Delete Edit Undo Red Black



- Co đồ thị G0 thành G1: ☐ Copy đồ thị G0

Đồ thị G1

Help Clear shift Delete Edit Undo Red Black



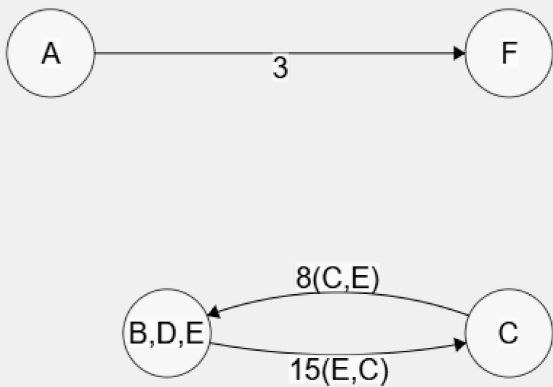
Lập 1:

- Xây dựng đồ thị xấp xỉ H1: ☐ Copy đồ thị G1

Đồ thị H1

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black

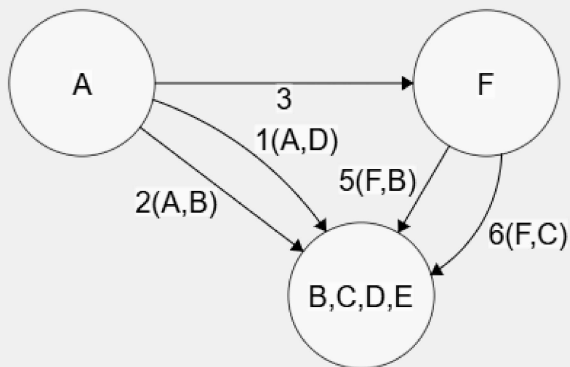


- Co đồ thị G1 thành G2: ☐ Copy đồ thị G1

Đồ thị G2

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black



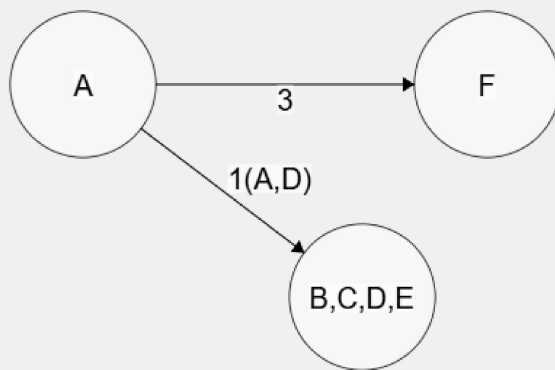
Lập 2:

- Xây dựng đồ thị xấp xỉ H2: ☐ Copy đồ thị G2

Đồ thị H2

☐ ☐ ☐ ☐

Help Clear shift Delete Edit Undo Red Black



II. Pha giãn

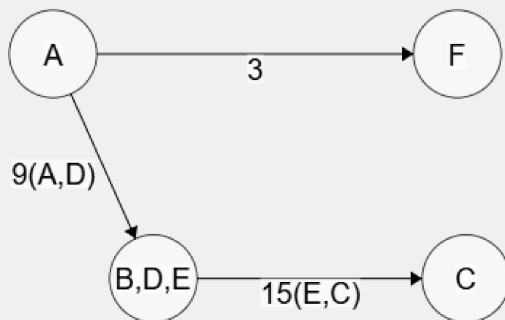
- T2 = H2

- Giãn T2 thành T1: ☐ Copy đồ thị T2

Cây khung T1



Help Clear shift Delete Edit Undo Red Black

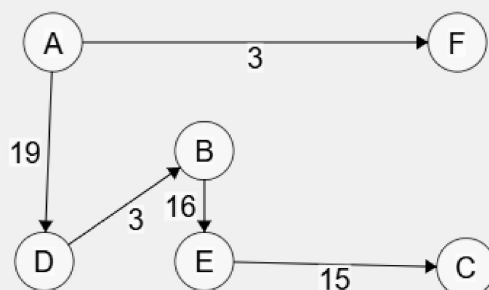


- Giãn T1 thành T0: ☐ Copy đồ thị T1

Cây khung T0



Help Clear shift Delete Edit Undo Red Black



	Test	Got	
✓	I. PHA CO II. PHA GIÃN	I. PHA CO Xây dựng đồ thị xấp xỉ H0: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị co G1: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị xấp xỉ H1: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị co G2: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 Xây dựng đồ thị xấp xỉ H2: - [I] Tất cả các bước okie. - [I] Điểm: 1.0/1.0 H2 không chứa chu trình => thoát vòng lặp, chuyển sang PHA GIÃN II. PHA GIÃN T2 = H2. Giãn cây T2 thành T1: - Tất cả các bước đều okie. - [I] Điểm: 1.0/1.0 Giãn cây T1 thành T0: - Tất cả các bước đều okie. - [I] Điểm: 1.0/1.0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00. Accounting for previous tries, this gives **0.90/1.00**.

Question 10

Correct

Mark 1.00 out of 1.00

Cho mạng với đỉnh phát $s = 1$ và đỉnh thu $t = 6$ được biểu diễn bằng đồ thị như bên dưới.

Hãy áp dụng thuật toán Ford - Fulkerson tìm luồng cực đại trong mạng. Thuật toán gồm 2 bước chính:

- Khởi tạo một luồng hợp lệ bất kỳ (thường là luồng có giá trị 0)
- Lập để tìm tăng luồng. Việc tìm đường tăng luồng được thực hiện bằng cách gán các đỉnh dùng **hàng đợi** (theo thuật toán Edmonds-Karp)

Sau bước gán nhãn, nếu tìm được đường tăng luồng, hãy vẽ lại mạng sau khi tăng luồng và tiếp tục. Ngược lại nếu không tìm được đường tăng luồng, thuật toán kết thúc, hãy cho biết lát cắt hẹp nhất (S, T) và giá trị luồng cực đại trong mạng.

Quy ước

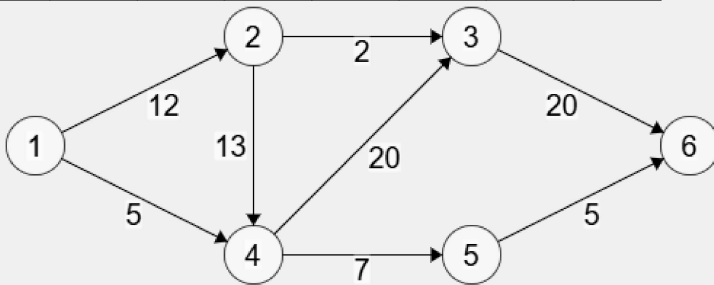
- Ghi nhãn các đỉnh theo mẫu: $(d[u], p[u], o[u])$ ví dụ: $(+, 1, oo)$, $(-, 4, 5)$
- Cột **Hàng đợi** ghi các đỉnh đang có trong hàng đợi, ngăn cách bằng dấu phẩy. Đầu hàng đợi (front) nằm bên trái. Ví dụ: 2, 4
- Ghi luồng trên cung theo mẫu: f/c hoặc f (ví dụ: $3/5$ hoặc 3) với f là luồng trên cung và c là khả năng thông qua của cung
- Khi xét đỉnh để gán nhãn, gán nhãn đỉnh liên quan đến **cung thuận trước, cung nghịch sau, đỉnh có thứ tự nhỏ trước, đỉnh có thứ tự lớn sau**

Answer: (penalty regime: 10, 20, ... %)

Reset answer

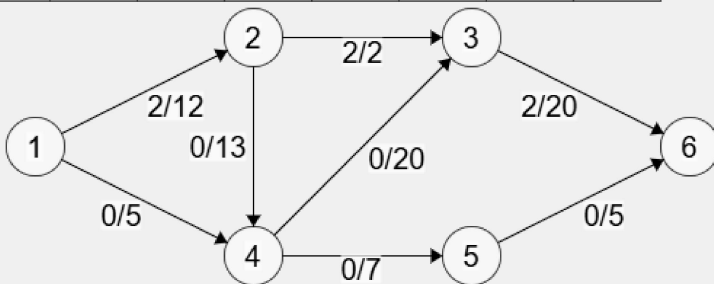
Mạng ban đầu

Help Clear shift Delete Edit Undo Red Black



I. Khởi tạo một luồng hợp lệ có giá trị không vượt quá 2

Help Clear shift Delete Edit Undo Red Black



II. Lập

Lần lặp 1

1.1 Gán nhãn các đỉnh dùng hàng đợi

	1	2	3	4	5	6	Hàng đợi
Khởi tạo	$(+, 1, oo)$						1
1		$(+, 1, 10)$		$(+, 1, 5)$			2, 4

	1	2	3	4	5	6	Hàng đợi
2							4
3			(+,4,5)		(+,4,5)		3,5
4						(+,3,5)	5,6
5							6

Add row

Delete row

☒ Tìm được đường tăng luồng

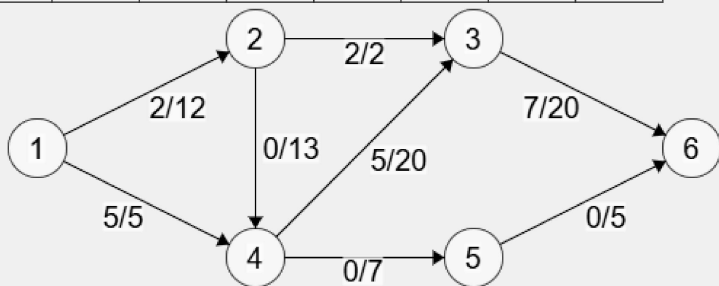
☐ Không có đường tăng luồng

- Đường tăng luồng (augmenting path): 1->4->3->6 ví dụ: 1 -> 3 -> 6
- Lượng luồng tăng thêm (bottle neck capacity): 5

1.2 Mạng sau khi tăng luồng

☐ Copy mạng ở bước trên

Help	Clear	shift	Delete	Edit	Undo	Red	Black
------	-------	-------	--------	------	------	-----	-------



Lần lặp 2

1.1 Gán nhãn các đỉnh dùng hàng đợi

	1	2	3	4	5	6	Hàng đợi
Khởi tạo	(+,1,∞)						1
1		(+,1,10)					2
2				(+,2,10)			4
3			(+,4,10)		(+,4,7)		3,5
4						(+,3,10)	5,6
5							6

Add row

Delete row

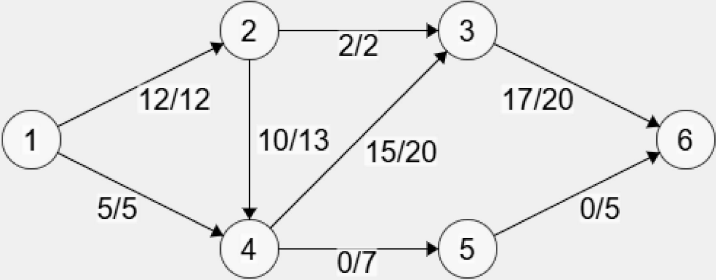
☒ Tìm được đường tăng luồng

☐ Không có đường tăng luồng

- Đường tăng luồng (augmenting path): 1->2->4->3->6 ví dụ: 1 -> 3 -> 6
- Lượng luồng tăng thêm (bottle neck capacity): 10

1.2 Mạng sau khi tăng luồng

☐ Copy mạng ở bước trên



Lần lặp 3

1.1 Gán nhãn các đỉnh dùng hàng đợi

	1	2	3	4	5	6	Hàng đợi
Khởi tạo	(+,1,∞)						1
1							
2							
3							
4							
5							

Add rowDelete row

☒ Tìm được đường tăng luồng

☐ Không có đường tăng luồng

III. Kết quả

- Luồng cực đại: 17
- Lát cắt hẹp nhất (S, T) tách s và t (ngăn cách các đỉnh bằng dấu phẩy, ví dụ: 1, 3, 4):
 - S = 1
 - T = 2,3,4,5,6

	Test	Got	
✓	I. Khởi tạo luồng (20%) II. Lập tìm đường tăng luồng (70%) III. Lát cắt hẹp nhất và giá trị luồng cực đại (10%) 3. Mạng sau khi tăng luồng (10%)	I. Khởi tạo luồng [I] Khởi tạo luồng okie. II. Lập Lần lặp 1 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. Lần lặp 2: [I] Nhãn của các đỉnh okie. Lần lặp 3: [I] Nhãn của các đỉnh okie. Lần lặp 4: [I] Nhãn của các đỉnh okie. 2. Đường tăng luồng và lượng luồng tăng thêm [I] Đường tăng luồng okie. [I] Lượng luồng tăng thêm okie. 3. Kiểm tra mạng sau khi tăng luồng Lần lặp 2 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. Lần lặp 2: [I] Nhãn của các đỉnh okie. Lần lặp 3: [I] Nhãn của các đỉnh okie. Lần lặp 4: [I] Nhãn của các đỉnh okie. 2. Đường tăng luồng và lượng luồng tăng thêm [I] Đường tăng luồng okie. [I] Lượng luồng tăng thêm okie. 3. Kiểm tra mạng sau khi tăng luồng Lần lặp 3 1. Kiểm tra việc gán nhãn (đánh dấu) các đỉnh Khởi tạo: [I] Nhãn của các đỉnh okie. Lần lặp 1: [I] Nhãn của các đỉnh okie. III. Kết quả (lát cắt và luồng cực đại) [I] Lát cắt hẹp nhất và giá trị luồng cực đại okie.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[◀ News forum](#)

Jump to...

